PROYECTO FINAL · SOLUCIONES PARA EL CLENA

Arquitectura

Entregable E5 · Modelo C4 (niveles 1 a 3), carpetas reales y decisiones técnicas

Nombre del equipo	Equipo P5
Proyecto elegido	P5 · Memoria Digital del Atlántico: digitalización, inventario y portal web del Archivo Histórico
Equipo de origen	Introducción a la Ingeniería, grupo 7: Santiago Barros, Iván Gómez, Samuel Pimienta y Juan Riaño
Repositorio	https://github.com/suscantillo/memoria-digital-atlantico
Tablero de GitHub Projects	https://github.com/users/suscantillo/projects/1
Integrantes	Andrea Carolina De la Ossa (Product Owner), Jesús David Cantillo Guerrero (Scrum Master), María Isabel Gutiérrez González y Alejandro David Alonso (Development Team)
Curso	Aplicaciones Web Backend · Universidad del Norte
Fecha y versión	13 de noviembre de 2026 · versión 1.0 · Entrega 1

Índice

1. Alcance 3
2. Nivel 1: contexto 3
3. Nivel 2: contenedores 4
4. Nivel 3: componentes de la API 5
5. Reglas entre capas 6
6. Carpetas reales 6
7. Una petición por las cuatro capas 6
8. Despliegue y arranque 9
9. Decisiones técnicas 10

Arquitectura

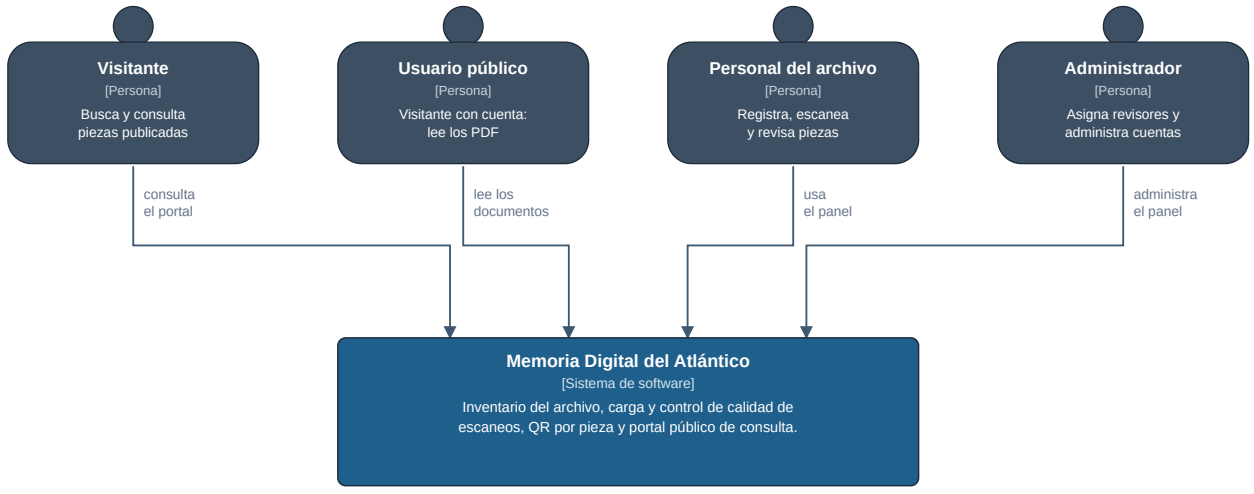
Memoria Digital del Atlántico · Entregable E5 · Aplicaciones Web Backend

1. Alcance

Este documento describe la arquitectura del sistema en la entrega 1 con el modelo C4: contexto, contenedores y componentes de la API. Después muestra las carpetas reales del repositorio, el recorrido de una petición por las cuatro capas con el código que la atiende, y las decisiones técnicas.

Parte	Tecnología	Para qué
API	NestJS 10 sobre Node 22, TypeScript 5	Rutas REST bajo `/api/v1`, validación, guardias y Swagger.
Persistencia	Prisma 6 y PostgreSQL 16	Esquema con migraciones versionadas; sin sincronización automática.
Interfaz	React 18, Vite 6, MUI 6, TanStack Query 5	Portal público y panel del personal en una sola SPA.
Despliegue	Docker Compose, nginx 1.27	Tres servicios: `db`, `backend` y `frontend`.
Calidad	Jest 29, ESLint, Prettier, GitHub Actions	Pruebas unitarias y e2e contra PostgreSQL real en cada PR.

2. Nivel 1: contexto



En la entrega 1 el sistema no depende de sistemas externos: guarda sus archivos en un volumen propio y no envía correos ni consulta servicios de terceros.

Figura 1. Quién usa el sistema y para qué.

3. Nivel 2: contenedores

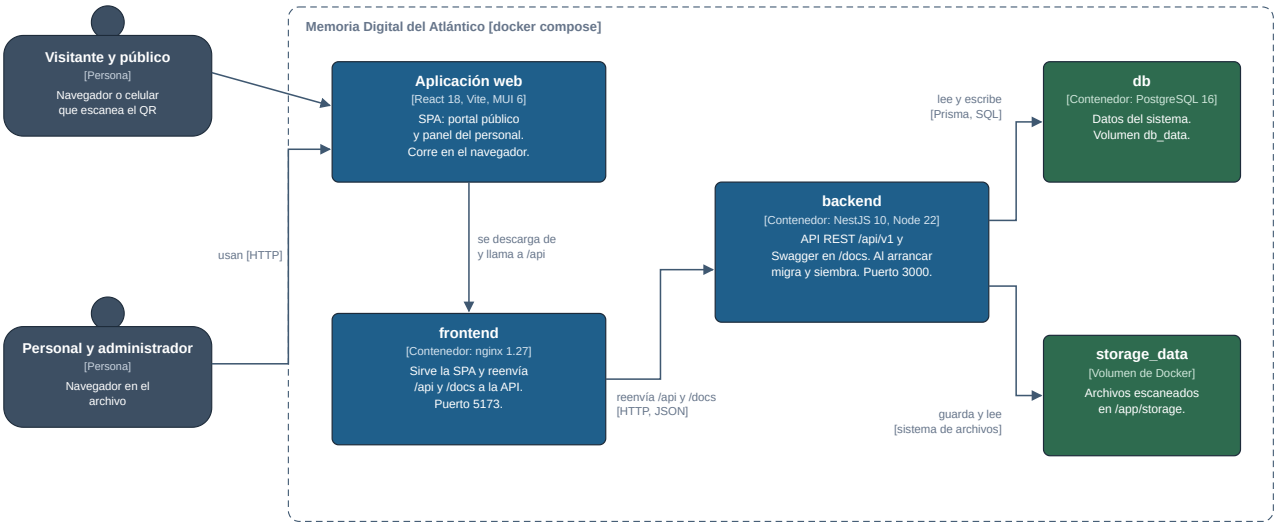
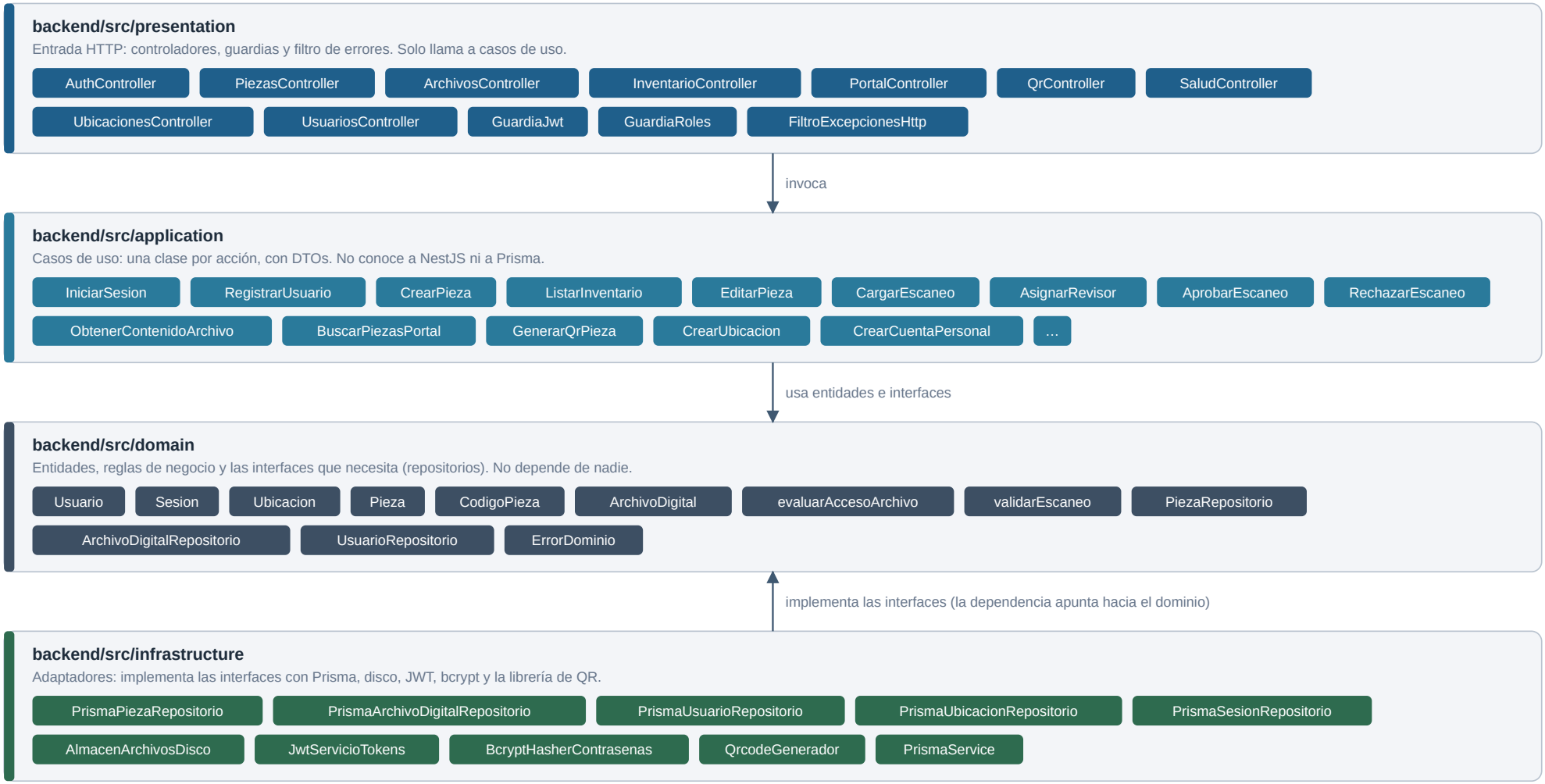


Figura 2. Lo que levanta docker compose up --build.

El navegador solo habla con el contenedor `frontend`: nginx entrega la aplicación y reenvía `/api` y `/docs` a la API, así el navegador ve un solo origen. El `backend` espera a que la base esté sana, aplica las migraciones, ejecuta la semilla y arranca. Los escaneos van a un volumen; la base guarda solo la ruta de cada archivo.

4. Nivel 3: componentes de la API



Cada módulo de NestJS (presentation/<módulo>/<módulo>.module.ts) une la interfaz con su implementación mediante un token y useFactory.

Figura 3. Las cuatro capas de backend/src y algunos de sus componentes reales.

5. Reglas entre capas

- `domain` no importa nada de `@nestjs`, `@prisma`, `class-validator` ni `class-transformer`.
- `application` no importa `@nestjs` ni `@prisma`: los casos de uso son clases planas que reciben interfaces por el constructor.
- Los controladores no tienen reglas de negocio: validan la entrada con el DTO, llaman al caso de uso y devuelven su resultado.
- Los errores de dominio (`ErrorRegla`, `ErrorConflicto`, `ErrorNoEncontrado`, `ErrorProhibido`, `ErrorNoAutorizado`) se traducen a HTTP en un solo lugar: `FiltroExcepcionesHttp`.
- Toda ruta nace protegida: las guardias de sesión y de rol son globales. Abrir una ruta exige marcarla con `@Publica()` o `@SesionOpcional()`, y una prueba e2e lleva la lista de las rutas abiertas.

Las dos primeras reglas se comprueban con una búsqueda en cada revisión de PR.

6. Carpetas reales

```
backend/src/domain/            entidades, reglas e interfaces de repositorio
    archivos  comun  piezas  sesiones  ubicaciones  usuarios
backend/src/application/       casos de uso y DTOs
    archivos  auth  comun  inventario  piezas  portal  qr  salud  ubicaciones  usuarios
backend/src/infrastructure/    Prisma, almacenamiento, JWT, bcrypt, QR, configuración
    archivos  auth  config  inventario  piezas  prisma  qr  salud  sesiones  ubicaciones  usuarios
backend/src/presentation/      controladores, módulos, guardias y filtro
    archivos  auth  comun  inventario  piezas  portal  qr  salud  ubicaciones  usuarios
backend/prisma/                schema.prisma, migraciones y semilla
    migrations  semilla
backend/test/                  pruebas e2e y dobles en memoria
    dobles
frontend/src/app/              router, tema y cliente de consultas
frontend/src/features/         una carpeta por funcionalidad: api, hooks, tipos, páginas
    archivos  auth  calidad  inventario  piezas  portal  qr  ubicaciones  usuarios
frontend/src/shared/           cliente de la API, sesión, componentes y layouts
    api  auth  componentes  layouts  tipos
docs/                          glosario, decisiones y los PDF de la entrega
    actas  plantillas
```

La API tiene 9 módulos de presentación: `auth`, `piezas`, `archivos`, `inventario`, `portal`, `qr`, `salud`, `ubicaciones`, `usuarios`. Cada uno tiene su controlador y su `*.module.ts`.

7. Una petición por las cuatro capas

El ejemplo es aprobar un escaneo en control de calidad: `POST /api/v1/archivos/:id/aprobar`. Los fragmentos son el código del repositorio, sin cambios.

1. presentation: el controlador

Las guardias ya identificaron al usuario y comprobaron su rol. El DTO exige los tres criterios; el controlador solo delega.

`backend/src/presentation/archivos/archivos.controller.ts`

```
  @Post(':id/aprobar')
  @Autenticado('ADMIN', 'PERSONAL')
  @HttpCode(HttpStatus.OK)
  @ApiOperation({ summary: 'Aprobar un escaneo EN_REVISION: la pieza queda PUBLICADA' })
  @ApiOkResponse({ type: ArchivoDigitalDto })
  @ApiForbiddenResponse({ description: 'Es PERSONAL y no es el revisor asignado.' })
  @ApiNotFoundResponse({ description: 'El archivo no existe.' })
  @ApiConflictResponse({ description: 'El escaneo ya fue revisado.' })
  aprobar(
    @Param('id', ParseUUIDPipe) id: string,
    @Body() _criterios: AprobarEscaneoDto,
    @UsuarioActual() usuario: UsuarioAutenticado,
  ): Promise<ArchivoDigitalDto> {
    return this.aprobarEscaneo.ejecutar(id, usuario);
  }
```

2. application: el caso de uso

Dice qué pasa al aprobar: el escaneo se aprueba, la pieza se publica y queda un evento. La parte común con rechazar está en la clase base.

`backend/src/application/archivos/aprobar-escaneo.use-case.ts`

```
export class AprobarEscaneo extends RevisarEscaneo {
  ejecutar(archivoId: string, quien: QuienRevisa): Promise<ArchivoDigitalDto> {
    return this.revisar(archivoId, quien, ({ archivo, pieza }) => {
      archivo.aprobar(quien.id);
      pieza.publicarPorEscaneoAprobado();
      return {
        accion: 'PUBLICADA',
        detalle: `Escaneo «${archivo.nombreOriginal}» aprobado: legible, completo y con datos correctos.`,
      };
    });
  }
}
```

3. application: lo común de aprobar y rechazar

Carga el escaneo con su pieza, comprueba que quien pregunta puede revisarlo, aplica la decisión y la guarda.

`backend/src/application/archivos/revisar-escaneo.use-case.ts`

```
  protected async revisar(
    archivoId: string,
    quien: QuienRevisa,
    decidir: (revision: RevisionEscaneo) => EventoRevision,
  ): Promise<ArchivoDigitalDto> {
    const revision = await this.archivos.buscarRevision(archivoId);
    if (!revision) {
      throw new ArchivoNoEncontradoError(archivoId);
    }
    // HU-23: el PERSONAL solo revisa lo que le asignaron (403)
    revision.archivo.exigirPuedeRevisar(quien);
    const evento = decidir(revision);

    const guardada = await this.archivos.guardarRevision(revision, evento);
    const detalle = await this.archivos.buscarDetalle(archivoId);
    if (!detalle) {
      throw new ArchivoNoEncontradoError(archivoId);
    }
    if (!guardada) {
      throw new ArchivoYaRevisadoError(detalle.estado);
    }
    return ArchivoDigitalDto.desde(detalle);
  }
```

4. domain: las reglas

La entidad decide quién puede revisar y exige que el escaneo siga en revisión.

`backend/src/domain/archivos/archivo-digital.entity.ts`

```
  exigirPuedeRevisar(quien: QuienRevisa): void {
    if (quien.rol === 'ADMIN') return;
    if (quien.rol === 'PERSONAL' && this.props.revisorAsignadoId === quien.id) return;
    throw new RevisionNoAsignadaError(this.props.revisorAsignadoId !== null);
  }

  aprobar(revisorId: string): void {
    this.exigirEnRevision();
    this.props.estado = 'APROBADO';
    this.props.motivoRechazo = null;
    this.marcarRevision(revisorId);
  }
```

5. infrastructure: el repositorio

Guarda escaneo, pieza y evento en una transacción. La condición sobre el estado impide que dos revisiones simultáneas se pisen.

`backend/src/infrastructure/archivos/prisma-archivo-digital.repositorio.ts`

```
  async guardarRevision(
    { archivo, pieza }: RevisionEscaneo,
    evento: EventoRevision,
  ): Promise<boolean> {
    return this.prisma.$transaction(async (tx) => {
      // la condición de estado hace de candado: de dos revisiones simultáneas solo entra una
      const cambio = await tx.archivoDigital.updateMany({
        where: { id: archivo.id, estado: 'EN_REVISION' },
        data: {
          estado: archivo.estado,
          motivoRechazo: archivo.motivoRechazo,
          revisadoPorId: archivo.revisadoPorId,
          revisadoEn: archivo.revisadoEn,
        },
      });
      if (cambio.count === 0) return false;

      await tx.pieza.update({ where: { id: pieza.id }, data: { estado: pieza.estado } });
      await tx.eventoPieza.create({
        data: {
          piezaId: pieza.id,
          usuarioId: archivo.revisadoPorId,
          accion: evento.accion,
          detalle: evento.detalle,
        },
      });
      return true;
    });
  }
```

6. presentation: el módulo que lo une

El módulo asocia la interfaz del dominio con la clase de Prisma mediante un token, y construye el caso de uso con una fábrica.

`backend/src/presentation/archivos/archivos.module.ts`

```
    { provide: ARCHIVO_DIGITAL_REPOSITORIO, useClass: PrismaArchivoDigitalRepositorio },
    { provide: REVISORES, useClass: PrismaRevisores },
    …
      provide: AprobarEscaneo,
      useFactory: (archivos: ArchivoDigitalRepositorio) => new AprobarEscaneo(archivos),
      inject: [ARCHIVO_DIGITAL_REPOSITORIO],
    },
```

8. Despliegue y arranque

Servicio	Imagen	Puerto	Depende de	Qué hace
`db`	postgres:16-alpine	5432	—	Base de datos, con comprobación de salud por `pg_isready`.
`backend`	node:22-alpine (multietapa)	3000	`db` sana	Al arrancar: `prisma migrate deploy`, semilla y `node dist/main.js`. Salud en `/api/v1/health`.

Servicio	Imagen	Puerto	Depende de	Qué hace
`frontend`	nginx:1.27-alpine	5173	`backend` sano	Sirve la SPA compilada y reenvía `/api` y `/docs`. Admite subidas de hasta 25 MB.

El proyecto arranca sin archivo `.env`: `docker-compose.yml` trae valores por defecto para desarrollo. La configuración se valida al iniciar (Joi) y la API no arranca si falta o está mal un valor.

9. Decisiones técnicas

El detalle y el porqué de cada una están en `docs/decisiones.md`.

N.º	Decisión	En una frase
1	Versiones fijadas	NestJS 10.4, Prisma 6, React 18.3, MUI 6, Vite 6 y ESLint 9 con flat config.
2	Casos de uso sin NestJS	Los casos de uso son clases planas
3	DTOs con class-validator y Swagger por plugin	Los DTOs viven en `application/` con decoradores de class-validator.
4	Errores de dominio traducidos en el filtro global	El dominio lanza `ErrorRegla`, `ErrorNoEncontrado`, `ErrorConflicto` y `ErrorNoAutorizado` sin saber de HTTP.
5	Unicidad de la terna por partida doble	`CrearUbicacion` consulta el repositorio antes de guardar y la tabla tiene restricción única sobre depósito, estantería y caja.
6	Sesiones con refresh JWT y `jti`	El token de acceso es un JWT corto.
7	bcryptjs en lugar de bcrypt	Mismo algoritmo y mismos hashes, sin compilación nativa
8	Pruebas e2e contra PostgreSQL real	`/health` comprueba la conexión de verdad, así que el e2e usa una base real.
9	docker-compose arranca sin `.env`	Los valores por defecto del compose son de desarrollo y permiten `docker compose up` sin pasos previos, como exige el curso.
10	nginx reenvía `/api` y `/docs`	El frontend y la API comparten origen en Docker, así que no hay CORS y Swagger queda accesible desde la misma URL.
11	Un solo chunk de terceros en Vite	Separar React de MUI y Emotion en chunks distintos produce una dependencia circular en la carga y la aplicación no arranca.
12	`GET /ubicaciones` público por ahora	La portada usa el listado de ubicaciones como prueba de conexión mientras no exista `/portal/piezas`.
13	La versión del aviso de privacidad la valida el backend	`POST /auth/registro` recibe `aceptaPrivacidad` y `versionPrivacidad`.
14	Fuentes autoalojadas	Archivo (títulos) y Figtree (texto) se instalan con Fontsource y se sirven desde la propia app en lugar de enlazar Google Fonts.
15	Las piezas restringidas no se encuentran por texto ni por década	`GET /portal/piezas` muestra las piezas restringidas solo con tipo y ubicación.
16	Aprobar el escaneo publica la pieza	En control de calidad una sola persona marca los tres criterios (legible, completa, datos correctos) y aprueba.
17	Una sola ruta entrega los escaneos y decide según quién pregunta	`GET /archivos/:id/contenido` sirve al revisor, al visor del portal y a quien lee un PDF con su cuenta.
18	Quien sube un escaneo no lo revisa	HU-09 y HU-10 nombraban al mismo rol para subir y para revisar, así que un `PERSONAL` podía aprobar lo que él mismo cargó.